

Regularization & Cross-Validation

PART A • CONCEPTUAL FOUNDATION (THEORY)



1. What is Regularization in Machine Learning? Why is it needed?

Regularization is a technique that adds a penalty term to the loss function to discourage large coefficients and overly complex models. Complex models tend to **overfit** – they memorize noise in the training data and perform poorly on new data. Regularization keeps the model simple, reduces variance and improves generalization.

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \lambda \cdot \text{Penalty}(\beta)$$

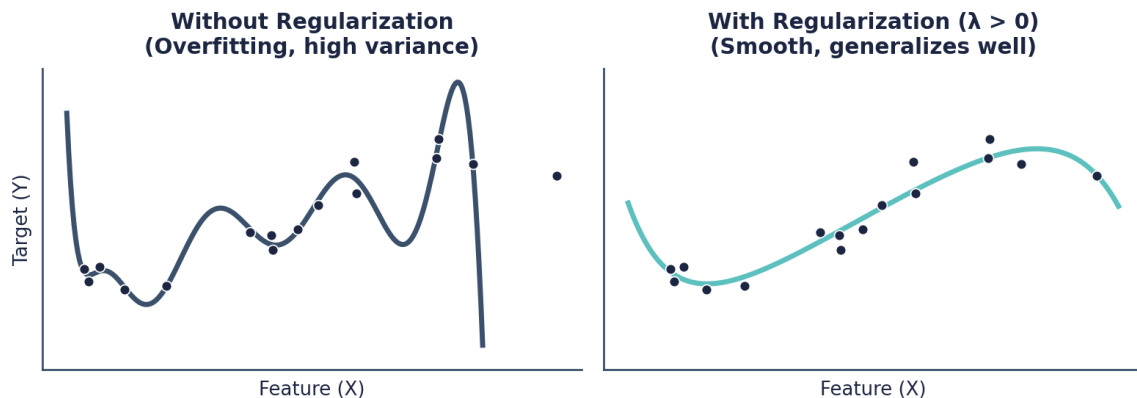


Figure 1: Without regularization the curve chases noise; with regularization it stays smooth.

2. Difference between Ridge Regression (L2) and Lasso Regression (L1)

Both add a penalty to linear regression, but **Ridge** penalises the squared size of coefficients while **Lasso** penalises their absolute size. This one difference changes how the two behave.

Feature	Ridge Regression (L2)	Lasso Regression (L1)
Penalty term	$\lambda \cdot \sum \beta_j^2$ (sum of squares)	$\lambda \cdot \sum \beta_j $ (sum of absolute values)
Effect on coefficients	Shrinks all coefficients towards zero, but never exactly zero	Can shrink some coefficients to exactly zero
Feature selection	No – keeps every feature	Yes – automatic feature selection
Best used when	Many correlated features, all somewhat useful	Many features, only a few truly matter (sparse)

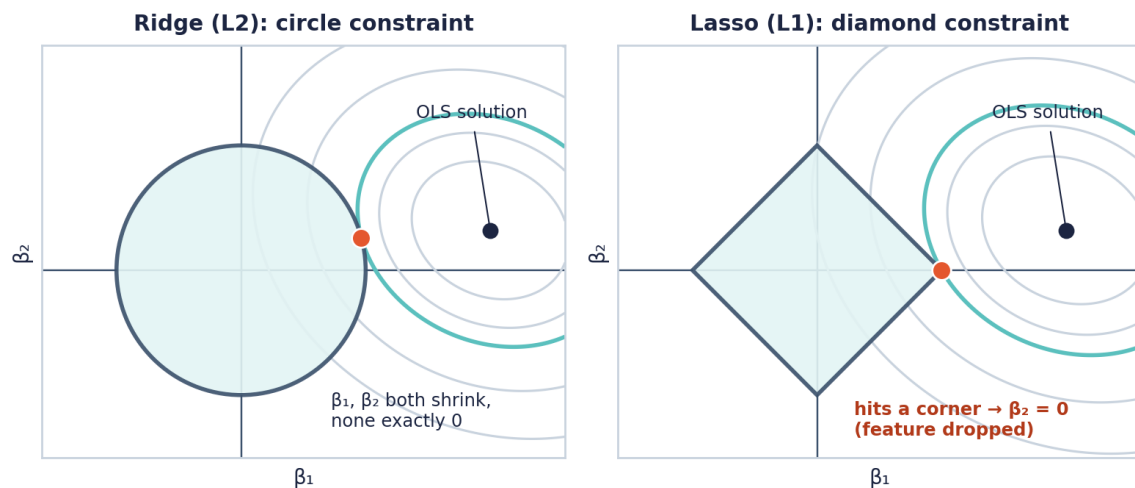


Figure 2: The Lasso diamond has corners on the axes, so its solution often lands where a coefficient = 0.

Tip: Always scale features (StandardScaler) before using Ridge or Lasso, because the penalty depends on coefficient size.

3. What is Cross-Validation and why is it important?

Cross-Validation is a resampling technique that splits data into several training and validation parts, trains the model on some, tests on the rest, repeats this, and averages the scores. It is important because it gives a **more reliable estimate** of performance than a single train-test split, helps detect overfitting and is used for hyperparameter tuning (e.g. choosing λ).

4. Cross-Validation Techniques

K-Fold Cross-Validation

Data is divided into **K equal folds**. Each fold is used once for validation while the other $K-1$ folds train the model. The final score is the average of K runs (usually $K = 5$ or 10).

Data split into K = 5 equal folds						
Iteration 1	Validation	Train	Train	Train	Train	Score 1
Iteration 2	Train	Validation	Train	Train	Train	Score 2
Iteration 3	Train	Train	Validation	Train	Train	Score 3
Iteration 4	Train	Train	Train	Validation	Train	Score 4
Iteration 5	Train	Train	Train	Train	Validation	Score 5

Figure 3: K-Fold ($K = 5$) – every fold becomes the validation set exactly once.

Stratified K-Fold Cross-Validation

Same as K-Fold, but every fold keeps the **same class proportion** as the full dataset. Best for classification, especially with imbalanced classes.

Leave-One-Out Cross-Validation (LOOCV)

A special case where **K = n**: one single observation is the test set each time and the rest train the model. Low bias and good for small datasets, but computationally expensive for large data.

Time Series Split

Respects **time order**: the training window grows forward and validation always comes after it, with no shuffling. This prevents future data leakage, so it is used for forecasting problems.

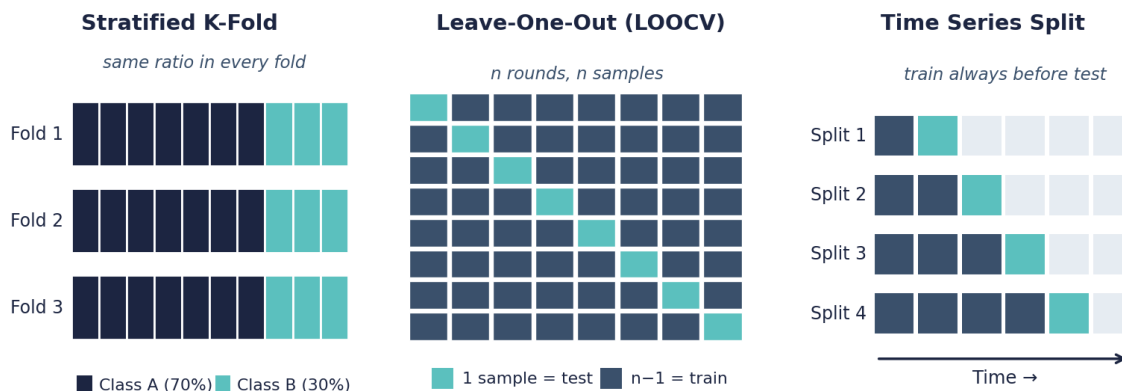


Figure 4: Stratified K-Fold, LOOCV and Time Series Split at a glance.

5. Why are tree-based models less sensitive to feature scaling?

Tree-based models (Decision Tree, Random Forest, XGBoost) split on one feature at a time using a threshold, e.g. $Income < 45,000$. They depend only on the **order** of values, not their magnitude. Scaling is a monotonic transformation, so the best split gives the same partition of data. Distance or gradient based models (KNN, SVM, Linear Regression, Neural Networks) do need scaling.

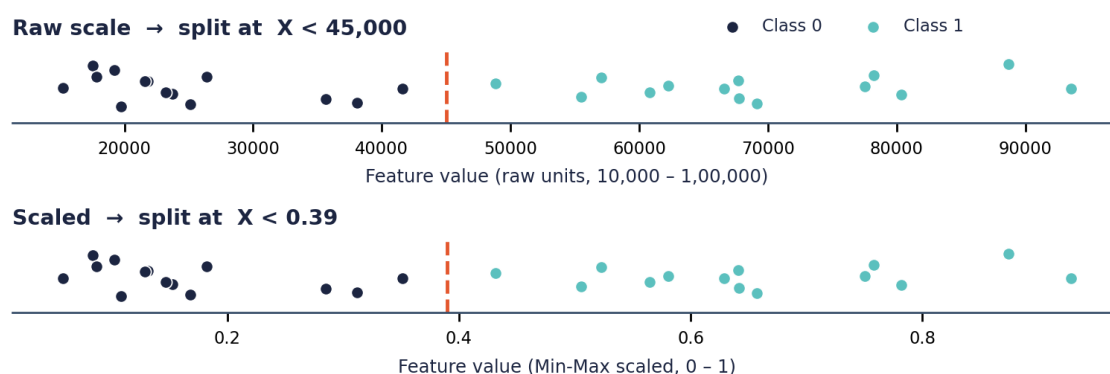


Figure 5: The same points are separated identically before and after scaling – only the threshold value changes.

Key Takeaway: Scaling is essential for Ridge / Lasso / KNN / SVM, but not required for tree-based models.